CHAPTER 12

Threads, Fibers, and Ractors

Being able to do more than one thing at the same time is pretty useful. For a computer program, being able to multitask means that if the program has to wait for one task to finish, like an API call to a slow server, it can turn control over to another task so as to get useful work done even while it waits. Being able to multitask can also mean that on a computer that has more than one CPU—which, these days, means a computer—then the program can split tasks across multiple CPUs. You can achieve tremendous speed boosts this way.

Being able to do more than one thing at a time is also pretty complicated for a computer program. When a program multitasks, another task can change the state of the data one task is using without that task doing anything, so its understanding of the data may not be correct. When a program multitasks, its tasks may fight for access to limited resources, like the filesystem, and might even overwrite each others changes. People are notoriously bad at predicting the effects of even mildly complicated threading scenarios, so unexpected bug cases are a real problem.

When writing programs that are doing multiple things at once, each "thing" is called a *thread*, and the goal is to have *thread safety*, meaning the code will execute correctly no matter in what order the threads operate. In cases where the order of operation matters—for example, if two threads are writing to a log file and instead of appending to the log file, the last thread overwrites the first one—this is called a *race condition* and is bad.

A key to achieving thread safety is to avoid having data or status information shared between threads, especially if that data is changeable by one thread without the knowledge of the other. Sometimes, though, you must share information, like access to a common database. In that case, you need constructs that limit access to shared resources such that only one thread can access them at a time.

Historically, Ruby programs have a Global Interpreter Lock (GIL), which insures that only one thread is actually being executed by Ruby at any time. The GIL is one way Ruby protects thread safety—since only one thread can run at a time, shared global resources within Ruby are automatically protected from being changed behind your thread's back. You still get the advantage of allowing one thread to take over execution if other threads are blocked. But you can't take advantage of, say, multiple parallel CPUs with a single Ruby interpreter (with the exception of the Ractor library). Ruby installations that do want to take advantage of multiple CPUs typically run multiple Ruby interpreters that communicate via an external data source or message system. (There's one relatively new Ruby construct that works around this—called Ractors—which we'll talk about at the end of this chapter.)